

Institut national des sciences appliquées et de technologie

GL3 - Année universitaire 2025-2026

CO-DESIGN

LAB 1 : Programmation OpenCL

Optimisation matérielle appliquée à la multiplication matricielle

Membres du groupe

Nom	Prénom
Bahri	Amal
Khili	Karim
Chaouch	Tasnim
Ourabi	Dorra

16 avril 2026

Table des matières

Introduction	2
1 Caractéristiques des Périphériques Compatibles OpenCL	2
2 Partie A : Optimisation du Kernel de Multiplication Matricielle	2
2.1 Kernel Coalescé Non-Optimisé (<code>kernel_1.cl</code>)	2
2.2 Première Optimisation : Workgroup Tiling & Wider Data-Types (<code>kernel_4.cl</code>)	4
2.3 Deuxième Optimisation : 2D Register Blocking (<code>kernel_6.cl</code>)	6
2.4 Kernel Entièrement Optimisé (<code>kernel_7.cl</code>)	8
2.5 Récapitulatif des Performances — Partie A	8
3 Partie B : Exécution du Kernel sur Multiples Périphériques OpenCL	10
3.1 Performances Indépendantes ($N = 8192$)	10
3.2 Stratégie de Répartition	11
3.3 Implémentation du Split et Gains de Performance	12
3.4 Récapitulatif des Performances — Partie B	14
Conclusion	15

Introduction

L'objectif de ce laboratoire est d'explorer les techniques d'optimisation matérielles appliquées à un calcul intensif standard : la **multiplication matricielle**.

Dans un premier temps, nous détaillerons les caractéristiques de notre environnement matériel d'exécution. Ensuite, à travers la **Partie A**, nous évaluerons progressivement l'impact de différentes approches d'optimisation — tuilage en mémoire locale, utilisation de types vectoriels pour élargir la bande passante, et tuilage par registres — sur un kernel OpenCL, en comparaison avec un kernel aux accès mémoire uniquement coalescés. Enfin, la **Partie B** sera dédiée à l'étude de la répartition de cette charge de calcul au sein d'un système hétérogène impliquant à la fois un GPU dédié et un processeur graphique intégré, mettant en lumière l'importance d'une stratégie de répartition équilibrée.

1. Caractéristiques des Périphériques Compatibles OpenCL

Le PC sur lequel nous avons exécuté les kernels est équipé d'un **GPU NVIDIA GeForce RTX 3050 6GB Laptop** avec les spécifications matérielles suivantes :

Table 1 – Spécifications matérielles du GPU NVIDIA GeForce RTX 3050 6GB Laptop

Paramètre	Valeur
Mémoire globale	6 Go
Cache global	560 Ko (READ_WRITE_CACHE)
Ligne de cache globale	128 Octets
Mémoire locale	48 Ko
Mémoire constante	64 Ko
Unités de calcul (SMs)	20
Taille max. du work-group	1024
Taille max. d'un work-item	[1024, 1024, 64]
Taille du Warp	32

2. Partie A : Optimisation du Kernel de Multiplication Matricielle

2.1. Kernel Coalescé Non-Optimisé (`kernel_1.cl`)

Ce kernel calcule la multiplication matricielle en associant strictement un thread global à un élément de la matrice résultante C . En utilisant `get_global_id(0)` comme itérateur de ligne dans la boucle interne, les threads d'un même warp accèdent à des emplacements mémoire contigus. Cela introduit la **coalescence mémoire**, rendant les accès à la mémoire bien plus efficaces qu'une approche purement naïve.

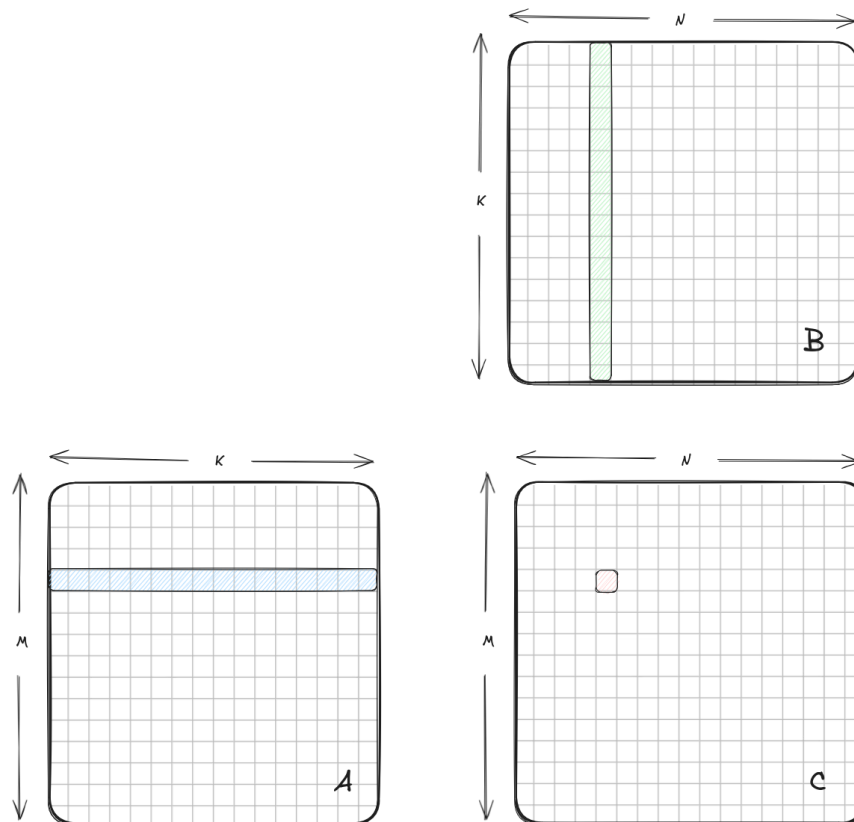


Figure 1 – Schéma d'exécution du kernel coalescé non-optimisé.

Performances (référence baseline) :

- Temps d'exécution : $\approx 37,8$ secondes
- Débit : **581 GFLOPS**

```
TS (4, 8, 16, 32) (default: 16):
Block Size is 16 * 16

Creating an OpenCL context...
Choose platform:
[0] <pyopencl.Platform 'NVIDIA CUDA' at 0x127062745a0>
[1] <pyopencl.Platform 'Intel(R) OpenCL Graphics' at 0x1270645d0a0>
[2] <pyopencl.Platform 'Intel(R) OpenCL' at 0x127103a2a10>
Choice [0]:
Choosing only available device: <pyopencl.Device 'NVIDIA GeForce RTX 3050 6GB Laptop GPU' on 'NVIDIA CUDA' at 0x12706274690>
Set the environment variable PYOPENCL_CTX='' to avoid being asked again.

Starting 20 OpenCL Matrix Multiplications...
End of 20 Matrix Multiplications...
37.80452013015747 seconds at 581.6826263052583 GFLOPS

Random samples of h_C:
h_C[60661409] = 152703.17
h_C[40967332] = 152703.17
h_C[1778848] = 152703.17
h_C[21608672] = 152703.17
h_C[33025037] = 152703.17
h_C[51672267] = 152703.17
h_C[60369966] = 152703.17
h_C[24565929] = 152703.17
h_C[43020267] = 152703.17
h_C[66016417] = 152703.17
cval: 152697.331712
```

Figure 2 – Résultats des performances pour le kernel coalescé de base.

2.2. Première Optimisation : Workgroup Tiling & Wider Data-Types (kernel_4.cl)

Ce kernel implémente deux optimisations pour contourner les limitations de la mémoire globale :

1. **Workgroup Tiling** : Les matrices A et B sont découpées en tuiles (sous-matrices). Les threads chargent ces tuiles de manière coopérative dans la **mémoire locale** partagée (`__local`). Chaque élément récupéré en mémoire globale est réutilisé plusieurs fois par d'autres threads au sein du même groupe de travail, réduisant drastiquement les défauts de cache et la latence de la mémoire globale.

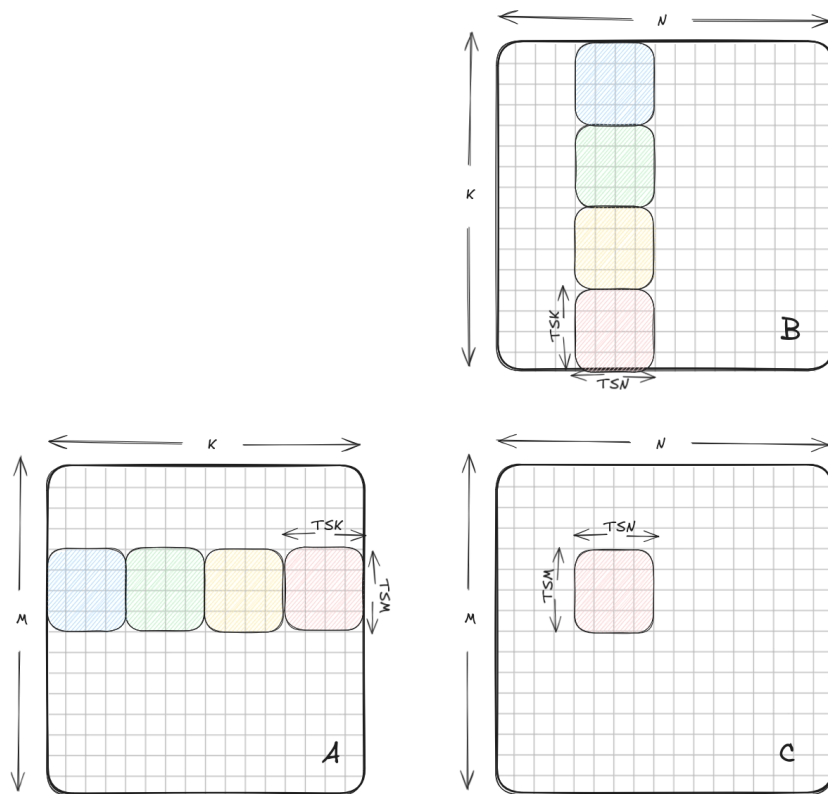


Figure 3 – Schéma d'explication du tuilage par mémoire locale.

2. **Wider Data-Types** : Au lieu de récupérer un flottant (`float`) à la fois, nous utilisons `float4` (un type vectoriel intégré de 4 flottants). Cela permet d'effectuer des transactions mémoire plus larges, augmentant ainsi la saturation du bus mémoire et parallélisant les chargements.

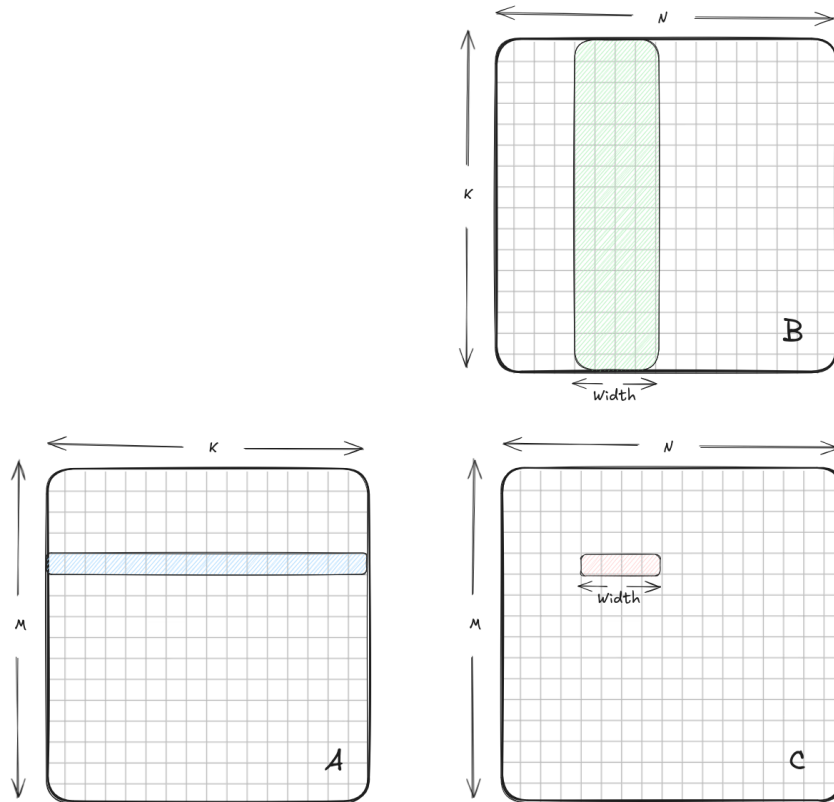


Figure 4 — Schéma d'explication de la vectorisation des accès.

Performances :

- Temps d'exécution : $\approx 22,47$ secondes
- Débit : **978 GFLOPS** — accélération de **1,68** $\times$ par rapport à la référence

```

Available kernels:
  0: Default coalsced kernel
  1: Wider data-types kernel
  2: Register blocking kernel
  3: Wider loads with register blocking kernel
  4: Prefetching kernel
Pick a kernel (default: 0): 1

TS (4, 8, 16, 32) (default: 16):
Block Size is 16 * 16
Work per thread (1, 2, 4) (default: 4):

Creating an OpenCL context...
Choose platform:
[0] <pyopencl.Platform 'NVIDIA CUDA' at 0x1aa4fd4d830>
[1] <pyopencl.Platform 'Intel(R) OpenCL Graphics' at 0x1aa26eb8a40>
[2] <pyopencl.Platform 'Intel(R) OpenCL' at 0x1aa59f184d0>
Choice [0]:
Choosing only available device: <pyopencl.Device 'NVIDIA GeForce RTX 3050 6GB Laptop GPU' on 'NVIDIA CUDA' at 0x1aa4fd4d2e0>
Set the environment variable PYOPENCL_CTX='' to avoid being asked again.

Starting 20 OpenCL Matrix Multiplications...
End of 20 Matrix Multiplications...
22.471111059188843 seconds at 978.6001456535812 GFLOPS

Random samples of h_C:
h_C[9932248] = 152703.17
h_C[52541678] = 152703.17
h_C[61050219] = 152703.17
h_C[42686152] = 152703.17
h_C[52819341] = 152703.17
h_C[43807775] = 152703.17
h_C[31914763] = 152703.17
h_C[39882530] = 152703.17
h_C[5675474] = 152703.17
h_C[15835690] = 152703.17
cval: 152697.331712

```

Figure 5 — Amélioration des performances suite au tuilage et à l'utilisation de types élargis.

2.3. Deuxième Optimisation : 2D Register Blocking (`kernel_6.cl`)

S'appuyer uniquement sur la mémoire locale entraîne tout de même une certaine latence. Le **2D register blocking** résout ce problème en faisant calculer à chaque thread plusieurs éléments de la matrice de sortie plutôt qu'un seul. En mettant en cache une sous-section de la tuile dans les **registres privés du GPU** — plus rapides que la mémoire locale —, nous réduisons de façon drastique le nombre d'instructions mémoire. De plus, chaque thread étant responsable du calcul d'un bloc 8×8 de valeurs de sortie ($WPTM = 8$, $WPTN = 8$), le ratio d'intensité arithmétique est considérablement augmenté.

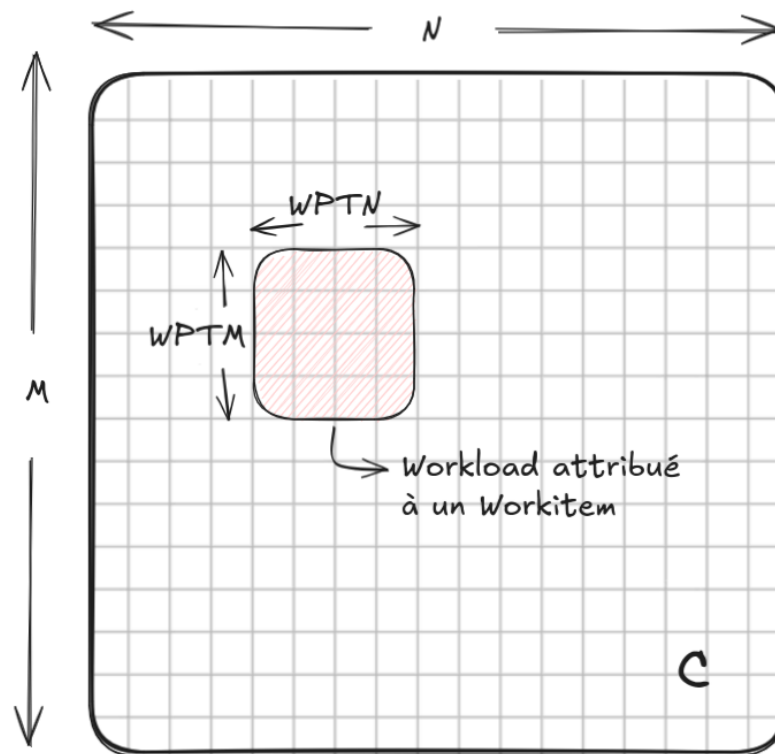


Figure 6 — Représentation du partitionnement par bloc en registres 2D.

Performances :

- Temps d'exécution : $\approx 6,01$ secondes
- Débit : **3658 GFLOPS** — accélération de **6,29 $\times$** par rapport à la référence

```

M (default: 8192):
N (default: 8192):
K (default: 8192):

Available kernels:
  0: Default coalesced kernel
  1: Wider data-types kernel
  2: Register blocking kernel
  3: Wider loads with register blocking kernel
  4: Prefetching kernel
Pick a kernel (default: 0): 2
TSN (default: 128):
TSN (default: 128):
TSK (default: 32):
WPTM (default: 8):
WPTN (default: 8):

Creating an OpenCL context...
Choose platform:
[0] <pyopencl.Platform 'NVIDIA CUDA' at 0x285b659f1d0>
[1] <pyopencl.Platform 'Intel(R) OpenCL Graphics' at 0x2858d86ac90>
[2] <pyopencl.Platform 'Intel(R) OpenCL' at 0x285c0930490>
Choice [0]:
Choosing only available device: <pyopencl.Device 'NVIDIA GeForce RTX 3050 6GB Laptop GPU' on 'NVIDIA CUDA' at 0x285b659f860>
Set the environment variable PYOPENCL_CTX='' to avoid being asked again.
C:\co-design\co-design\venv\Lib\site-packages\pyopencl\_init_.py:570: CompilerWarning: Non-empty compiler output encountered. Set the environment variable PYOPENCL_COMPILER_OUTPUT=1 to see more.
  lambda: self._prg.build(options_bytes, devices),

Starting 20 OpenCL Matrix Multiplications...
End of 20 Matrix Multiplications...
6.01016902923584 seconds at 3658.8376214630252 GFLOPS

```

Figure 7 — Amélioration des performances grâce au tuilage par registres.

2.4. Kernel Entièrement Optimisé (kernel_7.cl)

En combinant l'ensemble des approches précédentes, ce kernel exploite le **tuilage par Workgroup** en mémoire locale, couplé avec des **chargements élargis (float4)** et un **tuilage par Registres en 2D** (8×8 éléments calculés par thread). Les chargements vectorisés diminuent le nombre de transactions vers la mémoire locale, tandis que l'utilisation des registres permet de maintenir les pipelines de calcul occupés sans attendre les réponses du cache. Cette combinaison synergique offre un parallélisme optimal au niveau des instructions et une efficacité maximale des pipelines du GPU.

Performances :

- Temps d'exécution : $\approx 5,26$ secondes
- Débit : **4180 GFLOPS** — accélération de **7,19×** par rapport à la référence

```

Pick a kernel (default: 0): 3
Work per thread (1, 2, 4) (default: 4):
TSM (default: 128):
TSN (default: 128):
TSK (default: 32):
WPTM (default: 8):
WPTN (default: 8):

Creating an OpenCL context...
Choose platform:
[0] <pyopencl.Platform 'NVIDIA CUDA' at 0x2aa1fa31270>
[1] <pyopencl.Platform 'Intel(R) OpenCL Graphics' at 0x2aa29c81320>
[2] <pyopencl.Platform 'Intel(R) OpenCL' at 0x2aa1f932c00>
Choice [0]:
Choosing only available device: <pyopencl.Device 'NVIDIA GeForce RTX 3050 6GB Laptop GPU' on 'NVIDIA CUDA' at 0x2aa1fa312c0>
Set the environment variable PYOPENCL_CTX='' to avoid being asked again.

Starting 20 OpenCL Matrix Multiplications...
End of 20 Matrix Multiplications...
5.260453224182129 seconds at 4180.292385156403 GFLOPS

Random samples of h_C:
h_C[35291057] = 152703.17
h_C[31246784] = 152703.17
h_C[62806183] = 152703.17
h_C[23290603] = 152703.17
h_C[52945901] = 152703.17
h_C[29516383] = 152703.17
h_C[33846014] = 152703.17
h_C[57330678] = 152703.17
h_C[56267953] = 152703.17
h_C[27387380] = 152703.17
cval: 152697.331712
% (tp=1)

```

Figure 8 – Résultats de performance pour le kernel utilisant toutes les optimisations combinées.

2.5. Récapitulatif des Performances — Partie A

Table 2 – Évolution des performances selon les optimisations appliquées.

Kernel	Temps (s)	Débit (GFLOPS)	Accélération
kernel_1 — Coalescé (baseline)	$\approx 37,8$	581	1,00×
kernel_4 — Tiling + float4	$\approx 22,5$	978	1,68×
kernel_6 — Register Blocking 2D	$\approx 6,0$	3658	6,29×
kernel_7 — Toutes optimisations	$\approx 5,3$	4180	7,19×

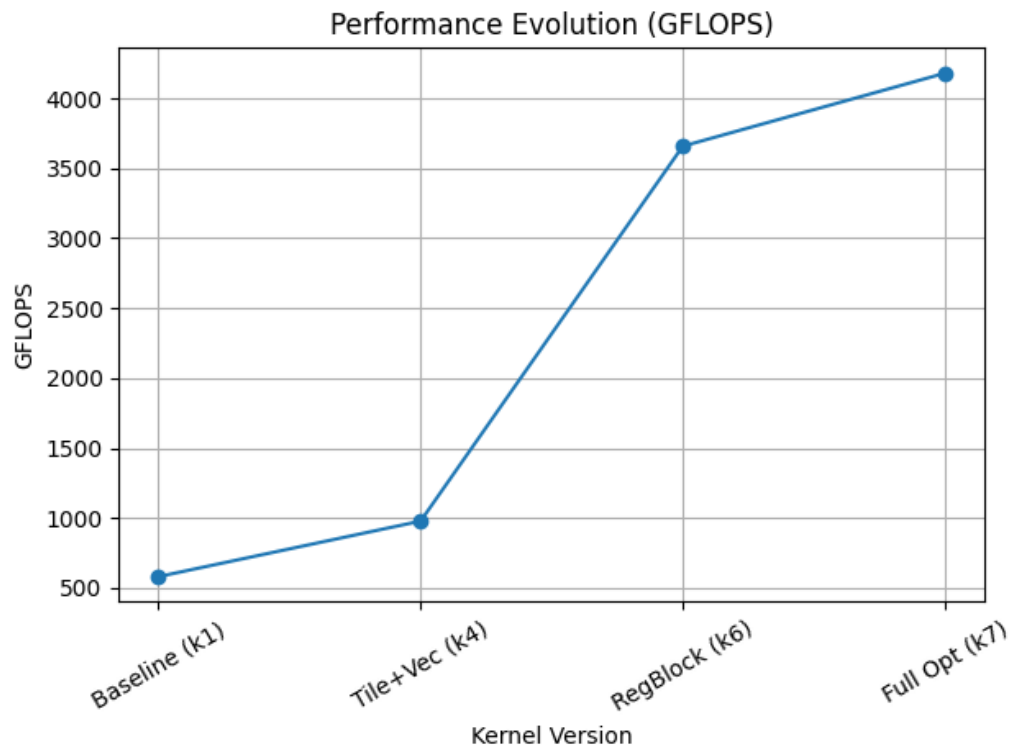


Figure 9 – Évolution du débit de calcul (en GFLOPS) en fonction des optimisations.

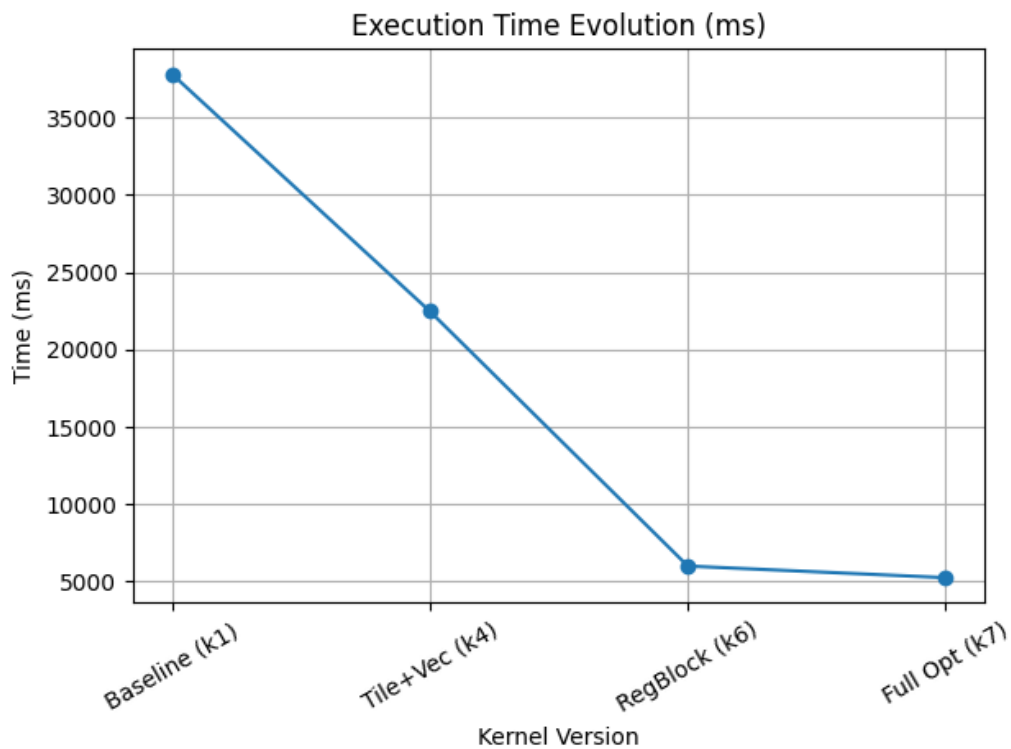


Figure 10 – Évolution du temps d'exécution (en millisecondes) selon les optimisations abordées.

3. Partie B : Exécution du Kernel sur Multiples Périphériques OpenCL

Dans cette partie, nous allons répartir le traitement entre la carte graphique dédiée (NVIDIA RTX 3050) et la puce graphique intégrée (iGPU). Le GPU dédié exécutera la logique de base lente et **non-coalescée**, tandis que le périphérique intégré exécutera l'approche la plus performante (le **kernel entièrement optimisé** construit tout au long de la Partie A).

3.1. Performances Indépendantes ($N = 8192$)

Afin de concevoir et justifier un ratio de répartition cohérent, nous avons d'abord comparé leurs performances lorsqu'ils opèrent de façon indépendante.

Table 3 – Comparaison de la puissance de calcul indépendante ($N = 8192$).

Périphérique	Méthode	Débit (GFLOPS)
GPU NVIDIA Dédié	Non-Coalescé	$\approx 120,29$ GFLOPS
GPU Intégré (iGPU)	Optimisé (kernel_7)	$\approx 128,60$ GFLOPS

```
Starting 1 OpenCL Matrix Multiplication(s)...
End of 1 Matrix Multiplication(s).
9.1405 seconds at 120.29 GFLOPS

Random samples of h_C:
  h_C[61980403] = 152703.171875
  h_C[58895923] = 152703.171875
  h_C[44516861] = 152703.171875
  h_C[26041137] = 152703.171875
  h_C[24873576] = 152703.171875
  h_C[36138116] = 152703.171875
  h_C[28752161] = 152703.171875
  h_C[31206848] = 152703.171875
  h_C[32253257] = 152703.171875
  h_C[63621114] = 152703.171875
  cval: 152697.331712
```

Figure 11 – Temps d'exécution (en millisecondes) du Nvidia GPU seulement.

```

Starting 1 OpenCL Matrix Multiplication(s)...
End of 1 Matrix Multiplication(s).
8.5498 seconds at 128.60 GFLOPS

Random samples of h_C:
    h_C[56234831] = 152703.171875
    h_C[7266427] = 152703.171875
    h_C[21010034] = 152703.171875
    h_C[59096560] = 152703.171875
    h_C[14172645] = 152703.171875
    h_C[11749010] = 152703.171875
    h_C[46262964] = 152703.171875
    h_C[37200908] = 152703.171875
    h_C[55301827] = 152703.171875
    h_C[11950852] = 152703.171875
    cval: 152697.331712

```

Figure 12 – Temps d'exécution (en millisecondes) du GPU intégré seulement.

Remarque : Du fait que le kernel de la carte NVIDIA repose sur des accès mémoire non-coalescés couplés à des matrices massives ($N = 8192$), son débit s'effondre littéralement ; cela rend ce traitement inéluctablement plus lent que le kernel optimisé s'exécutant sur un périphérique autrement considéré comme moins performant.

3.2. Stratégie de Répartition

Le but est de déléguer dynamiquement le calcul de certaines sous-matrices au travers des deux périphériques OpenCL de manière concurrente. Une stratégie de répartition de charge optimale se base sur une attribution du calcul strictement proportionnelle à la capacité de traitement réelle de chaque appareil.

Soient P_{GPU} les performances du GPU exécutant le code non-coalescé, et P_{iGPU} celles de l'iGPU exécutant le code optimisé. Pour que les deux périphériques terminent simultanément, les lignes M de la matrice doivent être séparées ainsi :

$$\text{Split}_{\text{iGPU}} = \frac{P_{\text{iGPU}}}{P_{\text{iGPU}} + P_{\text{GPU}}} \quad \text{Split}_{\text{GPU}} = \frac{P_{\text{GPU}}}{P_{\text{iGPU}} + P_{\text{GPU}}} \quad (1)$$

En prenant comme base nos mesures (≈ 120 GFLOPS contre ≈ 128 GFLOPS), la répartition optimale consiste à déléguer environ **51,6%** du travail à l'iGPU intégré et **48,4%** au GPU NVIDIA dédié.

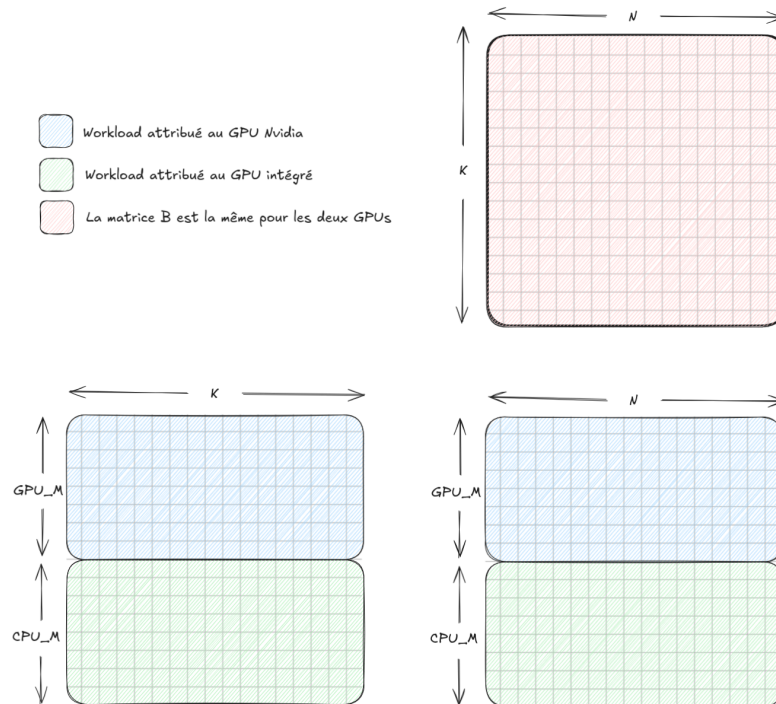


Figure 13 – Représentation visuelle de la stratégie de répartition de charge de travail.

3.3. Implémentation du Split et Gains de Performance

Une implémentation idéale de cette division, alignée rigoureusement aux proportions d'efficacité stipulées, entraîne un gain critique en capacité de débit. Cette démarche valorise la synergie d'exécution concurrente des deux files de requêtes contextuelles (*command queues*), qui peuvent masquer les temps de transfert des données en mémoire en les superposant à l'exécution simultanée du CPU et du GPU.

```
Starting 1 OpenCL Matrix Multiplication(s)...  
End of 1 Matrix Multiplication(s).  
4.8793 seconds at 225.34 GFLOPS  
  
Random samples of h_C:  
    h_C[31389978] = 152703.171875  
    h_C[40352404] = 152703.171875  
    h_C[44823587] = 152703.171875  
    h_C[6601467] = 152703.171875  
    h_C[24374182] = 152703.171875  
    h_C[49319953] = 152703.171875  
    h_C[9283846] = 152703.171875  
    h_C[64293242] = 152703.171875  
    h_C[27004328] = 152703.171875  
    h_C[31328407] = 152703.171875  
    cval: 152697.331712
```

Figure 14 – Performances accrues découlant de la coopération CPU (Optimisé) + GPU (Non-Coalescé).

3.4. Récapitulatif des Performances — Partie B

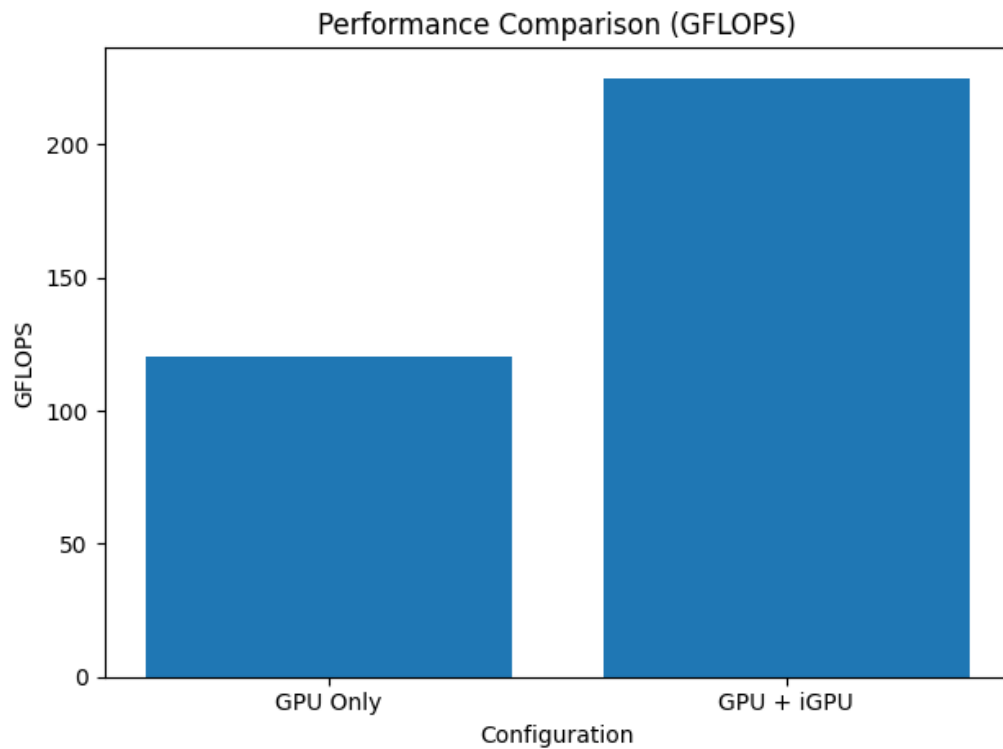


Figure 15 – Diagramme comparant le débit du mode combiné par rapport au mode isolé.

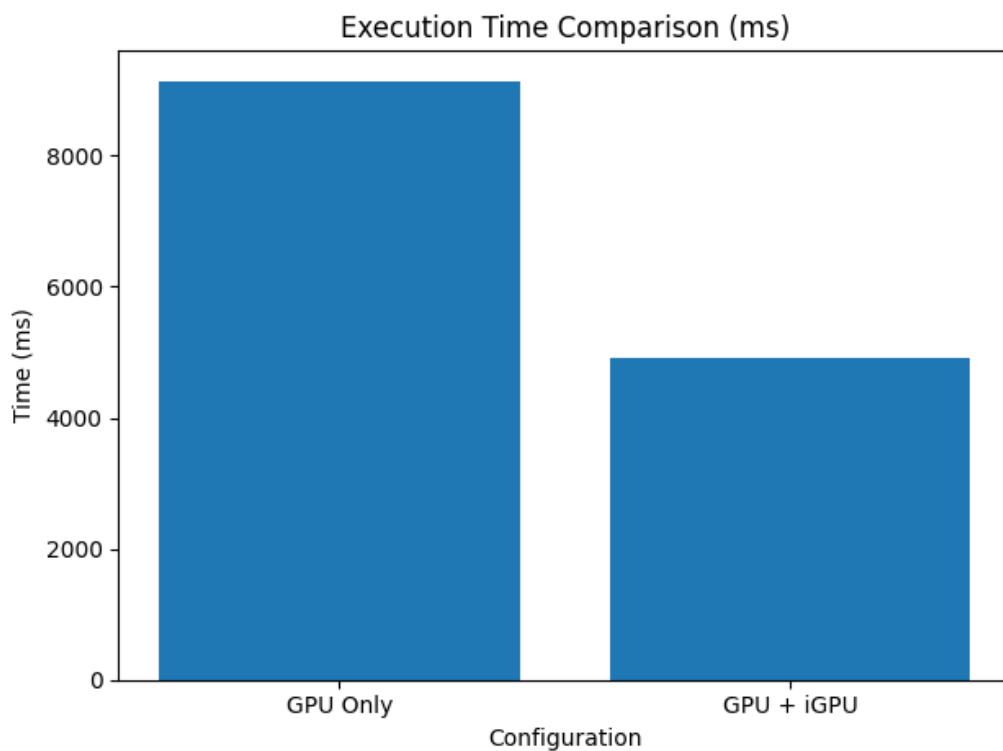


Figure 16 – Diagramme comparant le temps d'exécution du mode combiné par rapport au mode isolé.

Conclusion

En conclusion, la **Partie A** prouve que la simple coalescence des accès en mémoire principale n'exploite qu'une infime réserve du potentiel du GPU. L'utilisation stratégique des divers niveaux de la hiérarchie mémoire — mémoire locale partagée via le *workgroup tiling* et registres privés via le *2D register blocking* —, conjuguée à la vectorisation des accès, s'avère indispensable. Ces méthodes ont permis de consolider l'intensité arithmétique et d'accomplir une accélération du débit de traitement d'un facteur **7,19×** (de 581 GFLOPS à près de 4180 GFLOPS).

Par ailleurs, la **Partie B** met en exergue l'intérêt d'une co-exécution multi-périphériques, apte à capitaliser sur le parallélisme asynchrone des transferts mémoire (*Host-to-Device*) et du temps de calcul (*Kernel Exec*). Toutefois, un tel système hétérogène ne devient un levier d'efficacité qu'à la condition stricte d'intégrer une répartition des tâches pertinente et réaliste.